

Zero Downtime Postgres Database Decomposition

Overview

This runbook documents a **zero-downtime Postgres database decomposition strategy** using **Logical Replication** and **PgBouncer reconfiguration**. It enables the migration of a functional set of tables from a monolithic cluster to a dedicated Postgres cluster with a **controlled, reversible cutover** for both read and write traffic—ensuring continuous application availability.

This approach was successfully used to decompose the Security (Sec) application’s data from the Main cluster and now serves as a reusable framework for similar migrations.

Architecture Summary

- **Source Cluster:** The original Postgres cluster containing all application data prior to decomposition. In the Security (Sec) migration, this was the Main cluster—accessible via the `master.patroni` service endpoint for reads and writes, and `db-replica.service.consul` for read-only traffic.
- **Target Cluster:** The new Postgres cluster hosting the decomposed tables. In the Sec migration example, this was the Sec cluster—accessible via the `master.patroni-sec` service endpoint for reads and writes, and `sec-db-replica.service.consul` for read-only traffic.

Initially, applications connect through PgBouncer for all database read-write traffic via the Source Cluster’s RW Consul endpoint (`master.patroni`). Read traffic is routed to the Source Cluster’s read endpoint (`db-replica.service.consul`).

	Source Cluster	Target Cluster
Postgres Cluster	Main Cluster	Sec Cluster
PgBouncer Instances	pgbouncer, pgbouncer-sidekiq	pgbouncer-sec, pgbouncer-sidekiq-sec
RW Consul Endpoint	<code>master.patroni</code>	<code>master.patroni-sec</code>
Read Consul Endpoint	<code>db-replica.service.consul</code>	<code>sec-db-replica.service.consul</code>
Application Behavior	Connects through PgBouncer; RW via <code>master.patroni</code> , reads via <code>db-replica.service.consul</code>	Initially same as Source

Strategy Summary

This strategy uses **Logical Replication** to maintain real-time synchronization of selected tables during migration. It eliminates downtime by rerouting traffic through PgBouncer and validating at each stage.

Initial Setup

This phase prepares the **Target Cluster** and related infrastructure **before any traffic is switched to it**. The goal is to ensure the new environment is fully provisioned, replicating data from the **Source Cluster**, and ready to accept traffic in future phases—**without impacting the live application’s behavior or availability**.

- Provision the **Target Cluster** (e.g., `Sec cluster patroni-sec`).
 - Initialize it as a **physical standby** of the **Source Cluster** (e.g., `Main cluster patroni`).
 - Provision the **Target Cluster’s PgBouncer instances** (`pgbouncer-sec`, `pgbouncer-sidekiq-sec`). Initially, these PgBouncer instances are configured to connect to the **Source Cluster** via the `master.patroni` endpoint.
 - Configure the **Target Cluster’s application** to:
 - Route **read traffic** through the **Source Cluster’s** read endpoint: `db-replica.service.consul`.
 - Route **read-write traffic** through the **Target Cluster’s PgBouncer instances**, which are configured to connect to the **Source Cluster** via `master.patroni`.
-

Read Traffic Switchover

This phase transitions the **Target Cluster’s application** (e.g., `Sec Cluster application`) **read traffic** from the **Source Cluster** to the **Target Cluster’s** read replicas, allowing validation of the **Target Cluster’s** ability to serve production queries under real-world load. Importantly, write traffic continues to go through the **Source Cluster**, so this change is low-risk and reversible.

Preparation

- Prepare a merge request (MR) to update the application’s read endpoint from the **Source Cluster** (e.g., `db-replica.service.consul`) to the **Target Cluster** (e.g., `sec-db-replica.service.consul`).

Execution

1. Validate that **physical replication** from the **Source Cluster** to the **Target Cluster** is healthy (e.g., replication lag is within acceptable limits).
2. Merge the MR to update the application's read endpoint from the **Source Cluster** (e.g., `db-replica.service.consul`) to the **Target Cluster** (e.g., `sec-db-replica.service.consul`).
3. Validate that application **read traffic** is now routed to the **Target Cluster's** read replicas (`sec-db-replica.service.consul`).
4. Monitor application-level metrics (e.g., tuple fetches, index usage) and **PgBouncer** performance on the **Target Cluster**.
5. Conduct full QA and functional validation during a defined observation window to ensure application correctness and stability.

Rollback

- Revert the MR to restore the application's read endpoint to the **Source Cluster** (e.g., `db-replica.service.consul`).
 - Confirm that read queries are once again routed to the **Source Cluster**, and validate application behavior for correctness.
-

Full Cutover using Logical Replication

Preparation

- Convert the **Target Cluster** from **physical to logical replication**.
- Freeze DDL on all decomposed tables in the **Source Cluster** (e.g., **Main cluster**), if possible; otherwise, implement a full DDL freeze for the entire **Source Cluster** via a **feature flag**.

Read Traffic Switchover to Target Cluster (Logical Replicas)

- Follow the steps from **Read Traffic Switchover** to re-apply the read traffic migration, now targeting the **Target Cluster**, which is replicating data via **logical replication**.
 - Confirm the application continues to read from the **Target Cluster** (`sec-db-replica.service.consul`).
 - Observe application logs and Prometheus metrics to verify read traffic is routed to the **Target Cluster's** read replicas and validate correctness.
-

Write Traffic Switchover using PgBouncer Reconfiguration

1. **Disable Chef** and **PAUSE PgBouncers** on the **Target Cluster** (`pgbouncer-sec`, `pgbouncer-sidekiq-sec`).

2. Sync **Postgres sequences** for decomposed tables from the **Source Cluster** to the **Target Cluster**.
3. Ensure **Logical Replication lag is 0 bytes**.
4. Drop the **logical replication subscription** on the **Target Cluster**.
5. Apply **write-lock triggers** on the **Source Cluster** for all decomposed tables—ideally before creating reverse replication; if not feasible, apply them immediately after **RESUMEing PgBouncers**.
6. Create **reverse Logical Replication**:
 - Publication on the **Target Cluster**
 - Subscription on the **Source Cluster**
7. Update PgBouncer configs on the **Target Cluster** to point to its RW endpoint (`master.patroni-sec`) and reload PgBouncers.
8. **RESUME PgBouncers**.
9. Validate that application **write traffic**, in addition to previously switched **read traffic**, is now routed to the **Target Cluster** (`master.patroni-sec` and `sec-db-replica.service.consul`).
10. Merge **Chef MR** to persist PgBouncer configuration changes.
11. Sequentially run `chef-client` on PgBouncers.
12. Re-enable Chef on all affected nodes.

Automation: Use the `switchover.yml` Ansible playbook to automate most of the Read and Write Traffic Switchover steps. See the References section for the playbook repo and the related CR.

Post-Switchover Tasks

Post-Switchover QA Tests

- Run full E2E QA test suite against the decomposed environment.
- You may proceed with wrapping up the upgrade while monitoring test results in parallel.

Wrapping Up

- Remove any **silences**.
 - Set up **wal-g daily restore schedule** for the **Target Cluster**.
 - Ensure **Smoke tests** (automated via MR enabling `db_database_tasks` on k8s) and the **Full manual run** have both passed.
-

Close Rollback Window

Rollback Window: After switchover, a rollback window (e.g., 4 hours) allows reverting to the previous state without data loss, as logical replication continues replicating changes from the **Target Cluster** back to the **Source Cluster**. See the Rollback Plan for details.

Note: The rollback window ensures a safe reversion path, backed by reverse logical replication from the **Target Cluster** to the **Source Cluster**.

- Run the Ansible playbook to stop reverse logical replication: `stop_reverse_replication.yml`.
- On the **Source Cluster's** leader/writer node, drop the **subscription** (if still existing) for reverse logical replication.
- On the **Target Cluster's** leader/writer node, drop the **publication** and associated `logical_replication_slot` for reverse replication.
- Unfreeze DDL on all decomposed tables, preferably, otherwise, unfreeze DDL for the entire cluster via a **feature flag**.

Automation: Use the `stop_reverse_replication.yml` Ansible playbook to automate the safe closure of the rollback window. See the References section for the playbook repo.

Rollback Plan: Full Traffic Reversal

Read Rollback

1. Revert application configuration to use the **Source Cluster's** read endpoint (`db-replica.service.consul`).
2. Validate that read traffic routes correctly.
3. Confirm that application **read traffic** is now routed to the **Source Cluster's** read replicas (`db-replica.service.consul`).

RW Rollback (Write Traffic)

1. Disable Chef on PgBouncer hosts.
2. Drop **write-lock triggers** from the **Source Cluster** (Main).
3. **PAUSE** PgBouncers.
4. Sync **Postgres sequences** for all decomposed tables from the **Target Cluster** back to the **Source Cluster**.
5. Wait for **logical replication lag = 0 bytes**.
6. Drop the **reverse logical replication subscription** from the **Source Cluster**, and the corresponding **publication** from the **Target Cluster**.
7. Apply **write-lock triggers** on the **Target Cluster** for all decomposed tables. If not feasible, apply them immediately after **RESUMING** PgBouncers.
8. Update PgBouncer configuration to reconnect to the **Source Cluster's** RW endpoint (`master.patroni`).
9. **RESUME** PgBouncers.
10. Validate that application **write traffic**, along with previously switched **read traffic**, is now routed to the **Source Cluster** (`master.patroni` and `db-replica.service.consul`).
11. Merge the **Chef MR** to persist PgBouncer configuration changes.

12. Sequentially run `chef-client` on PgBouncer hosts.
13. Re-enable Chef on all affected nodes.

Automation: Use the `switchover_rollback.yml` Ansible playbook to automate most of the RW rollback steps. See the References section for the playbook repo.

Cleanup

- Merge **Terraform MR** to decommission the **Target Cluster**, if no longer needed.
-

Benefits of This Approach

- **Zero-downtime cutover** for both **read** and **write** traffic.
 - Staged and reversible migration process.
 - No Consul service endpoint manipulation required.
 - Avoids split-brain scenarios by using one-directional **Logical Replication** and **write-lock triggers**.
-

Future Enhancements

The following improvements are planned for a future iteration of this runbook:

- Add diagrams to visualize traffic flow, cluster roles, and switchover stages.
- Include application-specific steps tailored for the **Sec** application.
- Expand automation coverage for switchover steps and validation.

References

- Database Decomposition Strategies: Near Zero-Downtime Migration
- Security Database Decomposition Checklist
- Ansible Playbook – Convert Physical to Logical Replication
- Issue Template for Database Decomposition
- Sec DB Decomposition Change Request
- Runbooks - Patroni
- Runbooks - PgBouncer